# 🏠 Price Recommendation System using Machine Learning

## Complete End-to-End ML Pipeline for House Price Prediction

**Dataset:** Ames Housing Dataset
**Target Variable:** SalePrice
**Objective:** Build and compare multiple ML models to predict house prices accurately

---

## 📚 Step 1: Import Required Libraries

```
# Data manipulation and analysis
import pandas as pd
import numpy as np

# Data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning - Preprocessing
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_selection import SelectKBest, f_regression

# Machine Learning - Models
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR

# Machine Learning - Metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Model persistence
import joblib

# Ignore warnings for cleaner output
import warnings
warnings.filterwarnings('ignore')

# Set visualization style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

print("✅ All libraries imported successfully!")
```

```
✅ All libraries imported successfully!
```

## 📂 Step 2: Load and Explore Dataset

```
# Load the dataset
# Note: Update the file path as per your Colab environment
df = pd.read_csv('/content/train (1).csv')

print("="*80)
print("📊 DATASET LOADED SUCCESSFULLY")
print("="*80)
print(f"\nDataset Shape: {df.shape}")
print(f"Number of Rows: {df.shape[0]}")
print(f"Number of Columns: {df.shape[1]}")
```

```
================================================================================
📊 DATASET LOADED SUCCESSFULLY
================================================================================

Dataset Shape: (1460, 81)
Number of Rows: 1460
Number of Columns: 81
```

```
# Display first few rows
print("\n" + "="*80)
print("📋 FIRST 5 ROWS OF DATASET")
```

```
print("="*80)
df.head()
```

```
==============================

==============================
```

| Street | Alley | LotShape | LandContour | Utilities | ... | PoolArea | PoolQC | Fen |
|---|---|---|---|---|---|---|---|---|
| Pave | NaN | Reg | Lvl | AllPub | ... | 0 | NaN | N |
| Pave | NaN | Reg | Lvl | AllPub | ... | 0 | NaN | N |
| Pave | NaN | IR1 | Lvl | AllPub | ... | 0 | NaN | N |
| Pave | NaN | IR1 | Lvl | AllPub | ... | 0 | NaN | N |
| Pave | NaN | IR1 | Lvl | AllPub | ... | 0 | NaN | N |

```
# Display dataset information
print("\n" + "="*80)
print("ℹ️ DATASET INFORMATION")
print("="*80)
df.info()
```

```
 25  MasVnrType     588 non-null    object
 26  MasVnrArea     1452 non-null   float64
 27  ExterQual      1460 non-null   object
 28  ExterCond      1460 non-null   object
 29  Foundation     1460 non-null   object
 30  BsmtQual       1423 non-null   object
 31  BsmtCond       1423 non-null   object
 32  BsmtExposure   1422 non-null   object
 33  BsmtFinType1   1423 non-null   object
 34  BsmtFinSF1     1460 non-null   int64
 35  BsmtFinType2   1422 non-null   object
 36  BsmtFinSF2     1460 non-null   int64
 37  BsmtUnfSF      1460 non-null   int64
 38  TotalBsmtSF    1460 non-null   int64
 39  Heating        1460 non-null   object
 40  HeatingQC      1460 non-null   object
 41  CentralAir     1460 non-null   object
 42  Electrical     1459 non-null   object
 43  1stFlrSF       1460 non-null   int64
 44  2ndFlrSF       1460 non-null   int64
 45  LowQualFinSF   1460 non-null   int64
 46  GrLivArea      1460 non-null   int64
 47  BsmtFullBath   1460 non-null   int64
 48  BsmtHalfBath   1460 non-null   int64
 49  FullBath       1460 non-null   int64
 50  HalfBath       1460 non-null   int64
 51  BedroomAbvGr   1460 non-null   int64
 52  KitchenAbvGr   1460 non-null   int64
 53  KitchenQual    1460 non-null   object
 54  TotRmsAbvGrd   1460 non-null   int64
 55  Functional     1460 non-null   object
 56  Fireplaces     1460 non-null   int64
 57  FireplaceQu    770 non-null    object
 58  GarageType     1379 non-null   object
 59  GarageYrBlt    1379 non-null   float64
 60  GarageFinish   1379 non-null   object
 61  GarageCars     1460 non-null   int64
 62  GarageArea     1460 non-null   int64
 63  GarageQual     1379 non-null   object
 64  GarageCond     1379 non-null   object
 65  PavedDrive     1460 non-null   object
 66  WoodDeckSF     1460 non-null   int64
 67  OpenPorchSF    1460 non-null   int64
 68  EnclosedPorch  1460 non-null   int64
 69  3SsnPorch      1460 non-null   int64
 70  ScreenPorch    1460 non-null   int64
 71  PoolArea       1460 non-null   int64
 72  PoolQC         7 non-null      object
 73  Fence          281 non-null    object
 74  MiscFeature    54 non-null     object
 75  MiscVal        1460 non-null   int64
 76  MoSold         1460 non-null   int64
 77  YrSold         1460 non-null   int64
 78  SaleType       1460 non-null   object
 79  SaleCondition  1460 non-null   object
 80  SalePrice      1460 non-null   int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
```

```
# Display statistical summary
print("\n" + "="*80)
print("📈 STATISTICAL SUMMARY")
print("="*80)
df.describe()
```

```
================================================================================
📈 STATISTICAL SUMMARY
================================================================================
```

| | Id | MSSubClass | LotFrontage | LotArea | OverallQual | OverallCond | YearBuilt | YearRemodAdd | MasVnrArea |
|---|---|---|---|---|---|---|---|---|---|
| **count** | 1460.000000 | 1460.000000 | 1201.000000 | 1460.000000 | 1460.000000 | 1460.000000 | 1460.000000 | 1460.000000 | 1452.000000 |
| **mean** | 730.500000 | 56.897260 | 70.049958 | 10516.828082 | 6.099315 | 5.575342 | 1971.267808 | 1984.865753 | 103.685262 |
| **std** | 421.610009 | 42.300571 | 24.284752 | 9981.264932 | 1.382997 | 1.112799 | 30.202904 | 20.645407 | 181.066207 |
| **min** | 1.000000 | 20.000000 | 21.000000 | 1300.000000 | 1.000000 | 1.000000 | 1872.000000 | 1950.000000 | 0.000000 |
| **25%** | 365.750000 | 20.000000 | 59.000000 | 7553.500000 | 5.000000 | 5.000000 | 1954.000000 | 1967.000000 | 0.000000 |
| **50%** | 730.500000 | 50.000000 | 69.000000 | 9478.500000 | 6.000000 | 5.000000 | 1973.000000 | 1994.000000 | 0.000000 |
| **75%** | 1095.250000 | 70.000000 | 80.000000 | 11601.500000 | 7.000000 | 6.000000 | 2000.000000 | 2004.000000 | 166.000000 |
| **max** | 1460.000000 | 190.000000 | 313.000000 | 215245.000000 | 10.000000 | 9.000000 | 2010.000000 | 2010.000000 | 1600.000000 |

8 rows × 38 columns

```
# Check data types
print("\n" + "="*80)
print("🔍 DATA TYPES")
print("="*80)
print(df.dtypes.value_counts())
print("\nColumns by Data Type:")
print(f"Numerical Columns: {df.select_dtypes(include=[np.number]).shape[1]}")
print(f"Categorical Columns: {df.select_dtypes(include=['object']).shape[1]}")
```

```
================================================================================
🔍 DATA TYPES
================================================================================
object     43
int64      35
float64     3
Name: count, dtype: int64

Columns by Data Type:
Numerical Columns: 38
Categorical Columns: 43
```

## 🧹 Step 3: Data Cleaning

```
# Check for missing values
print("="*80)
print("🔍 MISSING VALUES ANALYSIS")
print("="*80)

missing_data = df.isnull().sum()
missing_percent = (missing_data / len(df)) * 100
missing_df = pd.DataFrame({
    'Missing_Count': missing_data,
    'Percentage': missing_percent
})
missing_df = missing_df[missing_df['Missing_Count'] > 0].sort_values('Missing_Count', ascending=False)

print(f"\nTotal columns with missing values: {len(missing_df)}")
print("\nTop 10 columns with most missing values:")
print(missing_df.head(10))
```

```
================================================================================
🔍 MISSING VALUES ANALYSIS
================================================================================

Total columns with missing values: 19

Top 10 columns with most missing values:
              Missing_Count  Percentage
PoolQC                 1453   99.520548
MiscFeature            1406   96.301370
Alley                  1369   93.767123
Fence                  1179   80.753425
```

```
MasVnrType              872   59.726027
FireplaceQu             690   47.260274
LotFrontage             259   17.739726
GarageType               81    5.547945
GarageYrBlt              81    5.547945
GarageFinish             81    5.547945
```

```
# Handle missing values
print("\n" + "="*80)
print("🔧 HANDLING MISSING VALUES")
print("="*80)

# For numerical columns: fill with median
numerical_cols = df.select_dtypes(include=[np.number]).columns
for col in numerical_cols:
    if df[col].isnull().sum() > 0:
        df[col].fillna(df[col].median(), inplace=True)
        print(f"✅ Filled {col} with median")

# For categorical columns: fill with mode (most frequent value)
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    if df[col].isnull().sum() > 0:
        df[col].fillna(df[col].mode()[0] if len(df[col].mode()) > 0 else 'Unknown', inplace=True)
        print(f"✅ Filled {col} with mode")

print(f"\n✅ Missing values after treatment: {df.isnull().sum().sum()}")
```

```
================================================================================
🔧 HANDLING MISSING VALUES
================================================================================
✅ Filled LotFrontage with median
✅ Filled MasVnrArea with median
✅ Filled GarageYrBlt with median
✅ Filled Alley with mode
✅ Filled MasVnrType with mode
✅ Filled BsmtQual with mode
✅ Filled BsmtCond with mode
✅ Filled BsmtExposure with mode
✅ Filled BsmtFinType1 with mode
✅ Filled BsmtFinType2 with mode
✅ Filled Electrical with mode
✅ Filled FireplaceQu with mode
✅ Filled GarageType with mode
✅ Filled GarageFinish with mode
✅ Filled GarageQual with mode
✅ Filled GarageCond with mode
✅ Filled PoolQC with mode
✅ Filled Fence with mode
✅ Filled MiscFeature with mode

✅ Missing values after treatment: 0
```

```
# Check for duplicates
print("\n" + "="*80)
print("🔍 CHECKING FOR DUPLICATES")
print("="*80)

duplicates = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicates}")

if duplicates > 0:
    df.drop_duplicates(inplace=True)
    print(f"✅ Removed {duplicates} duplicate rows")
else:
    print("✅ No duplicate rows found")

print(f"\nDataset shape after cleaning: {df.shape}")
```

```
================================================================================
🔍 CHECKING FOR DUPLICATES
================================================================================
Number of duplicate rows: 0
✅ No duplicate rows found

Dataset shape after cleaning: (1460, 81)
```

## 📊 Step 4: Outlier Detection and Treatment

```
# Select numerical columns for outlier detection (excluding Id and target)
numerical_features = df.select_dtypes(include=[np.number]).columns.tolist()
if 'Id' in numerical_features:
    numerical_features.remove('Id')
if 'SalePrice' in numerical_features:
    numerical_features.remove('SalePrice')

print("="*80)
print("🔍 OUTLIER DETECTION USING Z-SCORE METHOD")
print("="*80)

# Function to detect outliers using Z-score
def detect_outliers_zscore(data, threshold=3):
    z_scores = np.abs((data - data.mean()) / data.std())
    return z_scores > threshold

# Count outliers for each feature
outlier_counts_z = {}
for col in numerical_features[:10]:  # Check first 10 numerical features
    outliers = detect_outliers_zscore(df[col])
    outlier_counts_z[col] = outliers.sum()

print("\nOutliers detected (Z-score > 3):")
for col, count in sorted(outlier_counts_z.items(), key=lambda x: x[1], reverse=True)[:10]:
    print(f"{col}: {count} outliers")
```

```
================================================================================
🔍 OUTLIER DETECTION USING Z-SCORE METHOD
================================================================================

Outliers detected (Z-score > 3):
BsmtFinSF2: 50 outliers
MasVnrArea: 32 outliers
MSSubClass: 30 outliers
OverallCond: 28 outliers
LotFrontage: 16 outliers
LotArea: 13 outliers
YearBuilt: 6 outliers
BsmtFinSF1: 6 outliers
OverallQual: 2 outliers
YearRemodAdd: 0 outliers
```

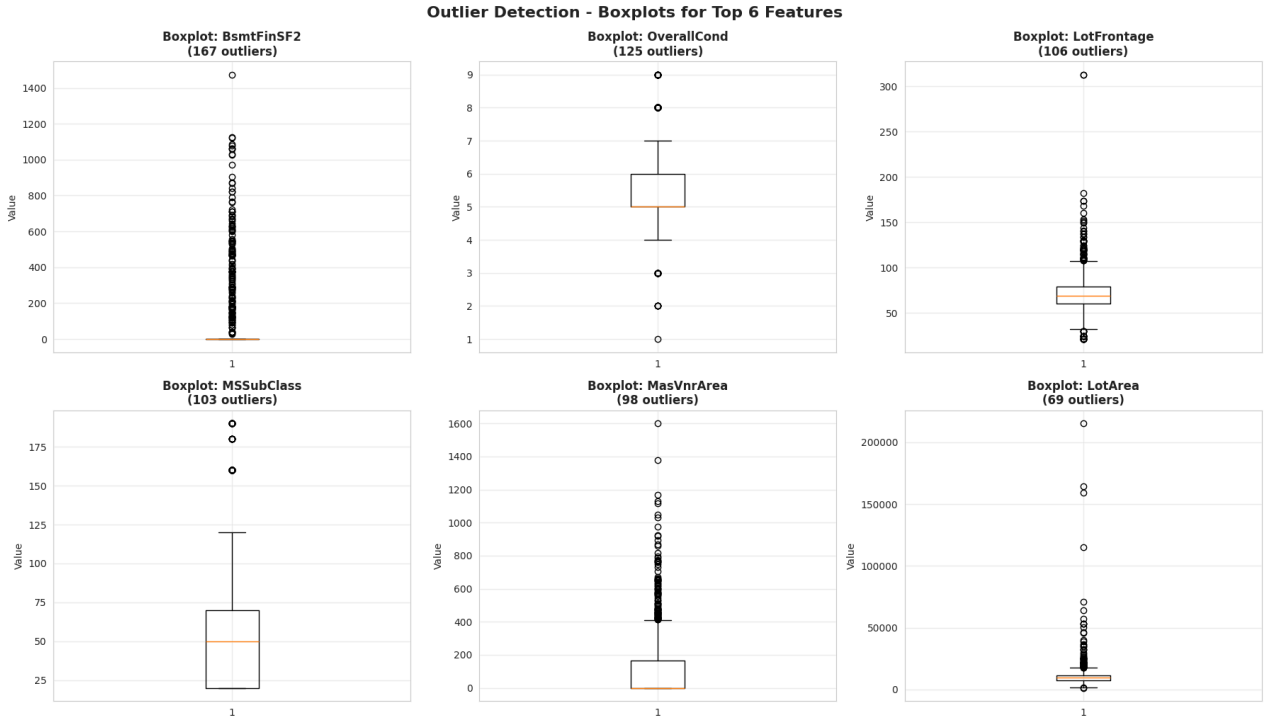
```
print("\n" + "="*80)
print("🔍 OUTLIER DETECTION USING IQR METHOD")
print("="*80)

# Function to detect outliers using IQR
def detect_outliers_iqr(data):
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return (data < lower_bound) | (data > upper_bound)

# Count outliers for each feature
outlier_counts_iqr = {}
for col in numerical_features[:10]:
    outliers = detect_outliers_iqr(df[col])
    outlier_counts_iqr[col] = outliers.sum()

print("\nOutliers detected (IQR method):")
for col, count in sorted(outlier_counts_iqr.items(), key=lambda x: x[1], reverse=True)[:10]:
    print(f"{col}: {count} outliers")
```

```
================================================================================
🔍 OUTLIER DETECTION USING IQR METHOD
================================================================================

Outliers detected (IQR method):
BsmtFinSF2: 167 outliers
OverallCond: 125 outliers
LotFrontage: 106 outliers
MSSubClass: 103 outliers
MasVnrArea: 98 outliers
LotArea: 69 outliers
YearBuilt: 7 outliers
BsmtFinSF1: 7 outliers
OverallQual: 2 outliers
YearRemodAdd: 0 outliers
```

```
# Visualize outliers using boxplots
print("\n" + "="*80)
```

```
print("📊 VISUALIZING OUTLIERS")
print("="*80)

# Select top 6 features with most outliers for visualization
top_outlier_features = sorted(outlier_counts_iqr.items(), key=lambda x: x[1], reverse=True)[:6]
top_features = [item[0] for item in top_outlier_features]

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.ravel()

for idx, col in enumerate(top_features):
    axes[idx].boxplot(df[col].dropna(), vert=True)
    axes[idx].set_title(f'Boxplot: {col}\n({outlier_counts_iqr[col]} outliers)', fontsize=12, fontweight='bold')
    axes[idx].set_ylabel('Value')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.suptitle('Outlier Detection - Boxplots for Top 6 Features', fontsize=16, fontweight='bold', y=1.02)
plt.show()

print("✅ Boxplots generated successfully")
```

```
================================================================================
📊 VISUALIZING OUTLIERS
================================================================================
```




```
✅ Boxplots generated successfully
```

```
# Treatment: Cap outliers using IQR method (Winsorization)
print("\n" + "="*80)
print("🔧 TREATING OUTLIERS (CAPPING METHOD)")
print("="*80)

def cap_outliers(data, col):
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap the values
    data[col] = np.where(data[col] < lower_bound, lower_bound, data[col])
    data[col] = np.where(data[col] > upper_bound, upper_bound, data[col])
    return data

# Apply capping to selected features with significant outliers
```

```
features_to_cap = [feat for feat, count in outlier_counts_iqr.items() if count > 50]

for col in features_to_cap:
    df = cap_outliers(df, col)
    print(f"✅ Capped outliers in {col}")

print(f"\n✅ Outlier treatment completed for {len(features_to_cap)} features")
```

```
================================================================================
🔧 TREATING OUTLIERS (CAPPING METHOD)
================================================================================
✅ Capped outliers in MSSubClass
✅ Capped outliers in LotFrontage
✅ Capped outliers in LotArea
✅ Capped outliers in OverallCond
✅ Capped outliers in MasVnrArea
✅ Capped outliers in BsmtFinSF2

✅ Outlier treatment completed for 6 features
```

## 🔧 Step 5: Feature Engineering

```
print("="*80)
print("🔧 FEATURE ENGINEERING")
print("="*80)

# Create a copy for feature engineering
df_engineered = df.copy()

# Create new features
print("\n📊 Creating New Features...")

# Total Square Footage
if 'TotalBsmtSF' in df_engineered.columns and 'GrLivArea' in df_engineered.columns:
    df_engineered['TotalSF'] = df_engineered['TotalBsmtSF'] + df_engineered['GrLivArea']
    print("✅ Created TotalSF (Total Square Footage)")

# House Age
if 'YearBuilt' in df_engineered.columns:
    df_engineered['HouseAge'] = 2024 - df_engineered['YearBuilt']
    print("✅ Created HouseAge")

# Years Since Remodeling
if 'YearRemodAdd' in df_engineered.columns:
    df_engineered['YearsSinceRemod'] = 2024 - df_engineered['YearRemodAdd']
    print("✅ Created YearsSinceRemod")

# Total Bathrooms
bathroom_cols = ['FullBath', 'HalfBath', 'BsmtFullBath', 'BsmtHalfBath']
if all(col in df_engineered.columns for col in bathroom_cols):
    df_engineered['TotalBathrooms'] = (df_engineered['FullBath'] +
                                       df_engineered['HalfBath'] * 0.5 +
                                       df_engineered['BsmtFullBath'] +
                                       df_engineered['BsmtHalfBath'] * 0.5)
    print("✅ Created TotalBathrooms")

# Total Porch SF
porch_cols = ['WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch']
if all(col in df_engineered.columns for col in porch_cols):
    df_engineered['TotalPorchSF'] = (df_engineered['WoodDeckSF'] +
                                     df_engineered['OpenPorchSF'] +
                                     df_engineered['EnclosedPorch'] +
                                     df_engineered['3SsnPorch'] +
                                     df_engineered['ScreenPorch'])
    print("✅ Created TotalPorchSF")

print(f"\n✅ Feature Engineering Complete. New shape: {df_engineered.shape}")
```

```
================================================================================
🔧 FEATURE ENGINEERING
================================================================================

📊 Creating New Features...
✅ Created TotalSF (Total Square Footage)
✅ Created HouseAge
✅ Created YearsSinceRemod
✅ Created TotalBathrooms
✅ Created TotalPorchSF

✅ Feature Engineering Complete. New shape: (1460, 86)
```

```
# Encode categorical variables
print("\n" + "="*80)
print("🔤 ENCODING CATEGORICAL VARIABLES")
print("="*80)

# Get categorical columns (excluding Id)
categorical_columns = df_engineered.select_dtypes(include=['object']).columns.tolist()

print(f"\nTotal categorical columns: {len(categorical_columns)}")

# Label Encoding for categorical variables
label_encoders = {}
for col in categorical_columns:
    le = LabelEncoder()
    df_engineered[col] = le.fit_transform(df_engineered[col].astype(str))
    label_encoders[col] = le

print(f"\n✅ Encoded {len(categorical_columns)} categorical columns using Label Encoding")
print(f"Dataset shape: {df_engineered.shape}")
```

```
================================================================================
🔤 ENCODING CATEGORICAL VARIABLES
================================================================================

Total categorical columns: 43

✅ Encoded 43 categorical columns using Label Encoding
Dataset shape: (1460, 86)
```

```
# Feature Scaling will be done after train-test split to prevent data leakage
print("\n" + "="*80)
print("⚖️ FEATURE SCALING PREPARATION")
print("="*80)
print("\n✅ StandardScaler will be applied after train-test split to prevent data leakage")
print("This ensures the model doesn't learn from test data during scaling")
```

```
================================================================================
⚖️ FEATURE SCALING PREPARATION
================================================================================

✅ StandardScaler will be applied after train-test split to prevent data leakage
This ensures the model doesn't learn from test data during scaling
```

## 🎯 Step 6: Feature Selection

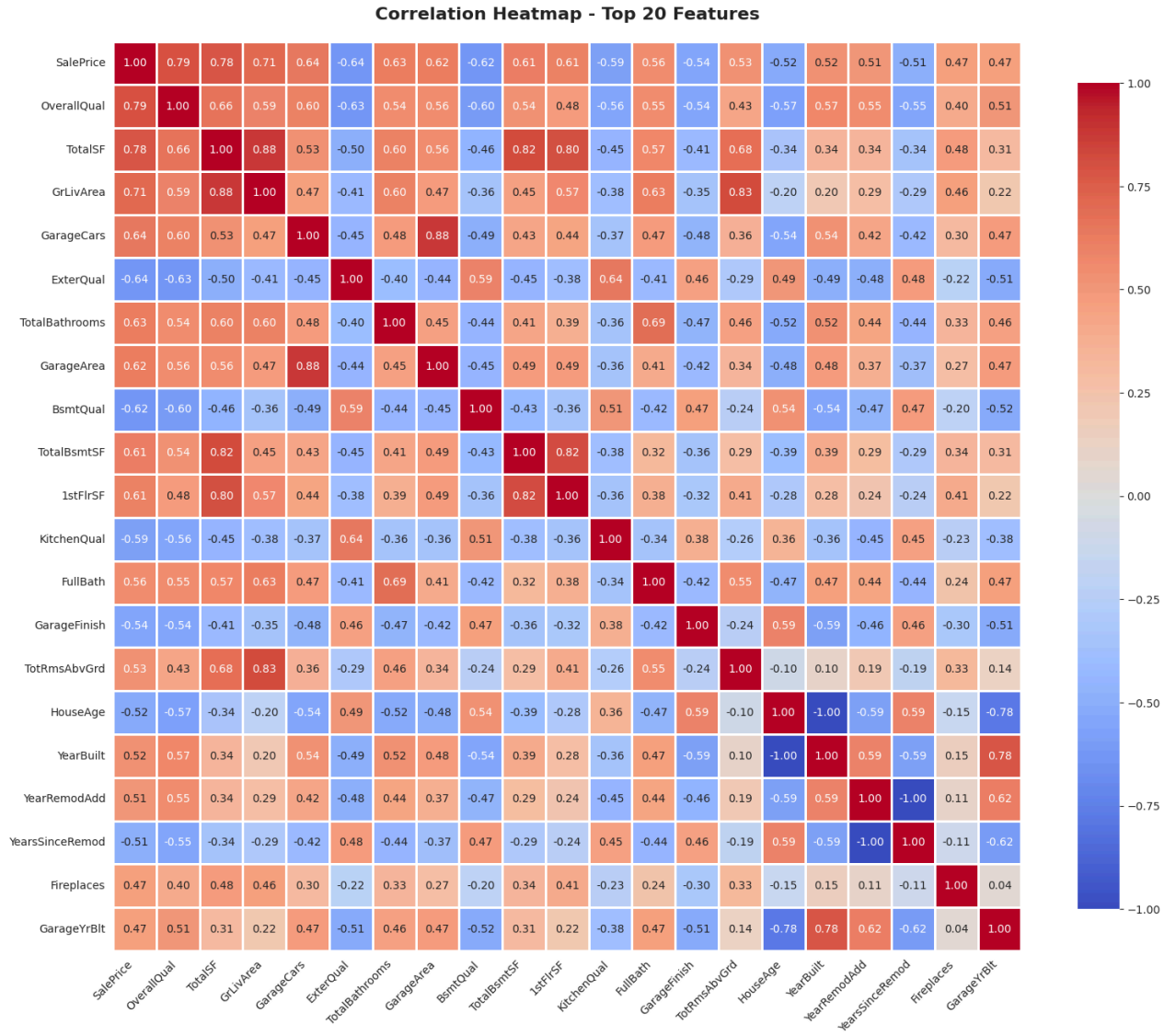
```
print("="*80)
print("🎯 FEATURE SELECTION")
print("="*80)

# Prepare data for correlation analysis
# Remove Id column if exists
if 'Id' in df_engineered.columns:
    df_analysis = df_engineered.drop('Id', axis=1)
else:
    df_analysis = df_engineered.copy()

# Calculate correlation with target variable
correlation_with_target = df_analysis.corr()['SalePrice'].sort_values(ascending=False)

print("\n📊 Top 15 Features Correlated with SalePrice:")
print(correlation_with_target.head(16))  # 16 because SalePrice itself is included

print("\n📊 Bottom 10 Features (Lowest Correlation):")
print(correlation_with_target.tail(10))
```

```
================================================================================
🎯 FEATURE SELECTION
================================================================================

📊 Top 15 Features Correlated with SalePrice:
SalePrice         1.000000
OverallQual       0.790982
TotalSF           0.778959
GrLivArea         0.708624
GarageCars        0.640409
TotalBathrooms    0.631731
GarageArea        0.623431
TotalBsmtSF       0.613581
1stFlrSF          0.605852
FullBath          0.560664
```

```
TotRmsAbvGrd      0.533723
YearBuilt         0.522897
YearRemodAdd      0.507101
Fireplaces        0.466929
GarageYrBlt       0.466754
MasVnrArea        0.448618
Name: SalePrice, dtype: float64

📊 Bottom 10 Features (Lowest Correlation):
BsmtExposure      -0.295407
GarageType        -0.302105
HeatingQC         -0.400178
YearsSinceRemod   -0.507101
HouseAge          -0.522897
GarageFinish      -0.537242
KitchenQual       -0.589189
BsmtQual          -0.618025
ExterQual         -0.636884
BsmtFinSF2              NaN
Name: SalePrice, dtype: float64
```

```
# Visualize correlation heatmap
print("\n" + "="*80)
print("📊 CORRELATION HEATMAP")
print("="*80)

# Select top correlated features for better visualization
top_features = correlation_with_target.abs().sort_values(ascending=False).head(21).index
correlation_matrix = df_analysis[top_features].corr()

plt.figure(figsize=(16, 14))
sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('Correlation Heatmap - Top 20 Features', fontsize=16, fontweight='bold', pad=20)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

print("✅ Correlation heatmap generated")
```

```
================================================================================
📊 CORRELATION HEATMAP
================================================================================
```




```
✅ Correlation heatmap generated
```

```
# Drop features with very low correlation (threshold: |correlation| < 0.05)
print("\n" + "="*80)
print("🔧 DROPPING LOW CORRELATION FEATURES")
print("="*80)

low_corr_threshold = 0.05
low_corr_features = correlation_with_target[abs(correlation_with_target) < low_corr_threshold].index.tolist()

# Remove SalePrice from the list if present
if 'SalePrice' in low_corr_features:
    low_corr_features.remove('SalePrice')

print(f"\nFeatures with correlation < {low_corr_threshold}: {len(low_corr_features)}")
print(f"Features to drop: {low_corr_features[:10]}...")  # Show first 10

# Drop low correlation features
df_selected = df_analysis.drop(columns=low_corr_features)

print(f"\n✅ Dropped {len(low_corr_features)} low correlation features")
print(f"Dataset shape after feature selection: {df_selected.shape}")
```

```
================================================================================
🔧 DROPPING LOW CORRELATION FEATURES
================================================================================

Features with correlation < 0.05: 14
Features to drop: ['MoSold', '3SsnPorch', 'Street', 'Fence', 'BsmtFinType2', 'MiscFeature', 'LandContour', 'Condition2', 'Ut

✅ Dropped 14 low correlation features
Dataset shape after feature selection: (1460, 71)
```

## 🎯 Step 7: Feature and Target Separation

```
print("="*80)
print("🎯 FEATURE AND TARGET SEPARATION")
print("="*80)

# Separate features (X) and target (y)
X = df_selected.drop('SalePrice', axis=1)
y = df_selected['SalePrice']

print(f"\nFeatures (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")
print(f"\nNumber of features: {X.shape[1]}")
print(f"Number of samples: {X.shape[0]}")

print("\n✅ Successfully separated features and target variable")
```

```
================================================================================
🎯 FEATURE AND TARGET SEPARATION
================================================================================

Features (X) shape: (1460, 70)
Target (y) shape: (1460,)

Number of features: 70
Number of samples: 1460

✅ Successfully separated features and target variable
```

## 🔀 Step 8: Train-Test Split

```
print("="*80)
print("🔀 TRAIN-TEST SPLIT")
print("="*80)

# Split the dataset (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"\nTraining set size: {X_train.shape[0]} samples ({(len(X_train)/len(X))*100:.1f}%)")
print(f"Testing set size: {X_test.shape[0]} samples ({(len(X_test)/len(X))*100:.1f}%)")

print(f"\nX_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\n✅ Train-Test split completed successfully")
```

```
================================================================================
🔀 TRAIN-TEST SPLIT
================================================================================

Training set size: 1168 samples (80.0%)
Testing set size: 292 samples (20.0%)

X_train shape: (1168, 70)
X_test shape: (292, 70)
y_train shape: (1168,)
y_test shape: (292,)

✅ Train-Test split completed successfully
```

```
# Apply Feature Scaling (after split to prevent data leakage)
print("\n" + "="*80)
print("⚖️ FEATURE SCALING")
print("="*80)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\n✅ StandardScaler applied to training and testing sets")
print("Note: Scaler was fitted on training data only to prevent data leakage")
```

```
================================================================================
⚖️ FEATURE SCALING
================================================================================

✅ StandardScaler applied to training and testing sets
Note: Scaler was fitted on training data only to prevent data leakage
```

## 🤖 Step 9: Baseline Model Training (Without Cross Validation)

```
print("="*80)
print("🤖 TRAINING MULTIPLE REGRESSION MODELS")
print("="*80)

# Initialize models
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),
    'KNN': KNeighborsRegressor(n_neighbors=5),
    'SVR': SVR(kernel='rbf'),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=100, random_state=42)
}

# Train all models
trained_models = {}

print("\n🏋️ Training models...\n")

for name, model in models.items():
    print(f"Training {name}...", end=' ')
    model.fit(X_train_scaled, y_train)
    trained_models[name] = model
    print("✅ Done")

print(f"\n✅ All {len(models)} models trained successfully!")
```

```
================================================================================
🤖 TRAINING MULTIPLE REGRESSION MODELS
================================================================================

🏋️ Training models...

Training Linear Regression... ✅ Done
Training Decision Tree... ✅ Done
Training Random Forest... ✅ Done
Training KNN... ✅ Done
Training SVR... ✅ Done
Training Gradient Boosting... ✅ Done

✅ All 6 models trained successfully!
```

## 🎯 Step 10: Making Predictions

```
print("="*80)
print("🎯 MAKING PREDICTIONS ON TEST DATA")
print("="*80)

# Store predictions
predictions = {}

print("\n📊 Generating predictions...\n")

for name, model in trained_models.items():
    y_pred = model.predict(X_test_scaled)
    predictions[name] = y_pred
    print(f"✅ {name}: Predictions generated")

print(f"\n✅ Predictions completed for all {len(trained_models)} models")
```

```
================================================================================
🎯 MAKING PREDICTIONS ON TEST DATA
================================================================================

📊 Generating predictions...

✅ Linear Regression: Predictions generated
✅ Decision Tree: Predictions generated
✅ Random Forest: Predictions generated
✅ KNN: Predictions generated
✅ SVR: Predictions generated
✅ Gradient Boosting: Predictions generated

✅ Predictions completed for all 6 models
```

## 📊 Step 11: Model Evaluation (Baseline)

```
print("="*80)
print("📊 MODEL EVALUATION - BASELINE PERFORMANCE")
print("="*80)

# Create results dataframe
results_baseline = []

for name, y_pred in predictions.items():
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    results_baseline.append({
        'Model': name,
        'MAE': mae,
        'MSE': mse,
        'RMSE': rmse,
        'R2 Score': r2
    })

# Create DataFrame and sort by R2 Score
baseline_df = pd.DataFrame(results_baseline)
baseline_df = baseline_df.sort_values('R2 Score', ascending=False)

print("\n" + "="*100)
print("📈 BASELINE MODEL PERFORMANCE METRICS")
print("="*100)
print(baseline_df.to_string(index=False))
```

```
print("\n" + "="*100)
print(f"🏆 BEST MODEL (Baseline): {baseline_df.iloc[0]['Model']}")
print(f"   R2 Score: {baseline_df.iloc[0]['R2 Score']:.4f}")
print(f"   RMSE: ${baseline_df.iloc[0]['RMSE']:,.2f}")
print("="*100)
```

```
================================================================================
📊 MODEL EVALUATION - BASELINE PERFORMANCE
================================================================================

====================================================================================================
📈 BASELINE MODEL PERFORMANCE METRICS
====================================================================================================
            Model          MAE          MSE         RMSE  R2 Score
Gradient Boosting 16382.163224 8.036052e+08 28347.931722  0.895232
    Random Forest 17517.369075 8.466103e+08 29096.569106  0.889625
    Decision Tree 23681.267123 1.241467e+09 35234.450718  0.838147
Linear Regression 21502.719406 1.256380e+09 35445.446073  0.836203
              KNN 21290.241096 1.310220e+09 36196.960480  0.829183
              SVR 59520.786062 7.855718e+09 88632.486634 -0.024170

====================================================================================================
🏆 BEST MODEL (Baseline): Gradient Boosting
   R2 Score: 0.8952
   RMSE: $28,347.93
====================================================================================================
```

## 🔄 Step 12: Cross Validation

```
print("="*80)
print("🔄 CROSS VALIDATION (K-Fold, k=5)")
print("="*80)

# Initialize K-Fold
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Store cross-validation results
cv_results = []

print("\n🔄 Performing 5-Fold Cross Validation...\n")

for name, model in models.items():
    print(f"Cross-validating {name}...", end=' ')

    # Perform cross-validation
    cv_scores = cross_val_score(model, X_train_scaled, y_train,
                                cv=kfold, scoring='r2')

    cv_results.append({
        'Model': name,
        'CV Mean R2': cv_scores.mean(),
        'CV Std R2': cv_scores.std(),
        'CV Min R2': cv_scores.min(),
        'CV Max R2': cv_scores.max()
    })

    print(f"✅ Mean R2: {cv_scores.mean():.4f} (±{cv_scores.std():.4f})")

# Create DataFrame
cv_df = pd.DataFrame(cv_results)
cv_df = cv_df.sort_values('CV Mean R2', ascending=False)

print("\n" + "="*100)
print("📊 CROSS VALIDATION RESULTS")
print("="*100)
print(cv_df.to_string(index=False))

print("\n✅ Cross-validation completed for all models")
```

```
================================================================================
🔄 CROSS VALIDATION (K-Fold, k=5)
================================================================================

🔄 Performing 5-Fold Cross Validation...

Cross-validating Linear Regression... ✅ Mean R2: 0.7985 (±0.0798)
Cross-validating Decision Tree... ✅ Mean R2: 0.6915 (±0.0873)
Cross-validating Random Forest... ✅ Mean R2: 0.8580 (±0.0418)
Cross-validating KNN... ✅ Mean R2: 0.7686 (±0.0337)
Cross-validating SVR... ✅ Mean R2: -0.0451 (±0.0145)
Cross-validating Gradient Boosting... ✅ Mean R2: 0.8854 (±0.0338)

====================================================================================================
```

```
📊 CROSS VALIDATION RESULTS
==========================================================================================
            Model  CV Mean R2  CV Std R2  CV Min R2  CV Max R2
Gradient Boosting    0.885392   0.033845   0.826423   0.912895
    Random Forest    0.857984   0.041780   0.791158   0.902506
Linear Regression    0.798518   0.079770   0.653224   0.861813
              KNN    0.768620   0.033685   0.726715   0.827144
    Decision Tree    0.691549   0.087309   0.527915   0.762797
              SVR   -0.045102   0.014475  -0.067994  -0.025179

✅ Cross-validation completed for all models
```

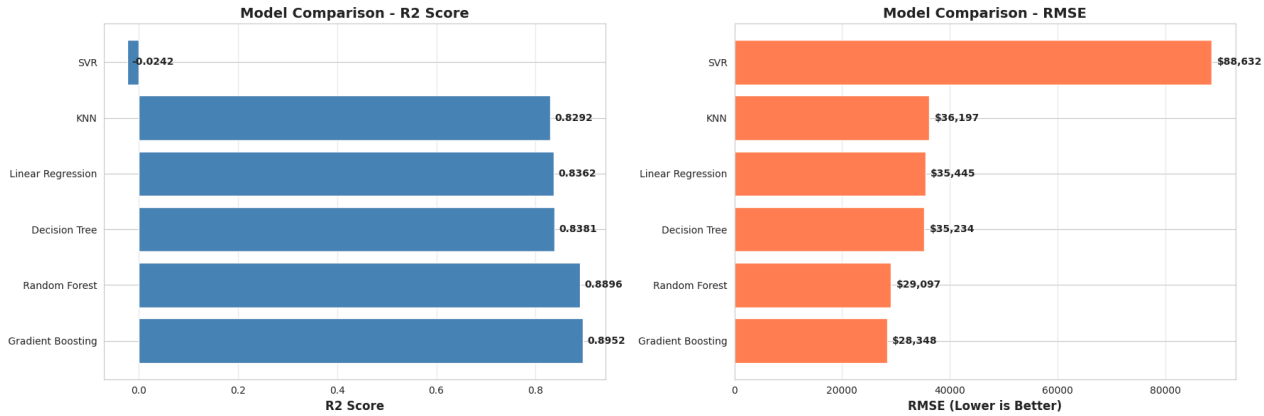
## 🏆 Step 13: Model Comparison & Ranking

```
print("="*80)
print("🏆 MODEL COMPARISON & RANKING")
print("="*80)

# Merge baseline and CV results
comparison_df = baseline_df.merge(cv_df, on='Model')
comparison_df = comparison_df.sort_values('R2 Score', ascending=False)

print("\n" + "="*120)
print("📊 COMPREHENSIVE MODEL COMPARISON")
print("="*120)
print(comparison_df.to_string(index=False))

# Find best model
best_model_name = comparison_df.iloc[0]['Model']
best_r2 = comparison_df.iloc[0]['R2 Score']
best_rmse = comparison_df.iloc[0]['RMSE']
best_cv_r2 = comparison_df.iloc[0]['CV Mean R2']

print("\n" + "="*120)
print("🥇 BEST MODEL")
print("="*120)
print(f"Model Name: {best_model_name}")
print(f"Test R2 Score: {best_r2:.4f}")
print(f"Test RMSE: ${best_rmse:,.2f}")
print(f"Cross-Validation Mean R2: {best_cv_r2:.4f}")
print("="*120)

# Model ranking
print("\n" + "="*80)
print("📋 MODEL RANKING (by R2 Score)")
print("="*80)
for idx, row in comparison_df.iterrows():
    rank = comparison_df.index.get_loc(idx) + 1
    print(f"{rank}. {row['Model']}: R2 = {row['R2 Score']:.4f}, RMSE = ${row['RMSE']:,.2f}")
```

```
================================================================================
🏆 MODEL COMPARISON & RANKING
================================================================================

========================================================================================================================
📊 COMPREHENSIVE MODEL COMPARISON
========================================================================================================================
            Model          MAE          MSE         RMSE  R2 Score  CV Mean R2  CV Std R2  CV Min R2  CV Max R2
Gradient Boosting 16382.163224 8.036052e+08 28347.931722  0.895232    0.885392   0.033845   0.826423   0.912895
    Random Forest 17517.369075 8.466103e+08 29096.569106  0.889625    0.857984   0.041780   0.791158   0.902506
    Decision Tree 23681.267123 1.241467e+09 35234.450718  0.838147    0.691549   0.087309   0.527915   0.762797
Linear Regression 21502.719406 1.256380e+09 35445.446073  0.836203    0.798518   0.079770   0.653224   0.861813
              KNN 21290.241096 1.310220e+09 36196.960480  0.829183    0.768620   0.033685   0.726715   0.827144
              SVR 59520.786062 7.855718e+09 88632.486634 -0.024170   -0.045102   0.014475  -0.067994  -0.025179

========================================================================================================================
🥇 BEST MODEL
========================================================================================================================
Model Name: Gradient Boosting
Test R2 Score: 0.8952
Test RMSE: $28,347.93
Cross-Validation Mean R2: 0.8854
========================================================================================================================

================================================================================
📋 MODEL RANKING (by R2 Score)
================================================================================
1. Gradient Boosting: R2 = 0.8952, RMSE = $28,347.93
2. Random Forest: R2 = 0.8896, RMSE = $29,096.57
3. Decision Tree: R2 = 0.8381, RMSE = $35,234.45
4. Linear Regression: R2 = 0.8362, RMSE = $35,445.45
5. KNN: R2 = 0.8292, RMSE = $36,196.96
6. SVR: R2 = -0.0242, RMSE = $88,632.49
```

```
# Visualize model comparison
print("\n" + "="*80)
print("📊 MODEL PERFORMANCE VISUALIZATION")
print("="*80)

fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# R2 Score Comparison
axes[0].barh(comparison_df['Model'], comparison_df['R2 Score'], color='steelblue')
axes[0].set_xlabel('R2 Score', fontsize=12, fontweight='bold')
axes[0].set_title('Model Comparison - R2 Score', fontsize=14, fontweight='bold')
axes[0].grid(axis='x', alpha=0.3)
for i, v in enumerate(comparison_df['R2 Score']):
    axes[0].text(v + 0.01, i, f'{v:.4f}', va='center', fontweight='bold')

# RMSE Comparison
axes[1].barh(comparison_df['Model'], comparison_df['RMSE'], color='coral')
axes[1].set_xlabel('RMSE (Lower is Better)', fontsize=12, fontweight='bold')
axes[1].set_title('Model Comparison - RMSE', fontsize=14, fontweight='bold')
axes[1].grid(axis='x', alpha=0.3)
for i, v in enumerate(comparison_df['RMSE']):
    axes[1].text(v + 1000, i, f'${v:,.0f}', va='center', fontweight='bold')

plt.tight_layout()
plt.show()

print("✅ Model comparison visualization complete")
```

```
================================================================================
📊 MODEL PERFORMANCE VISUALIZATION
================================================================================
```



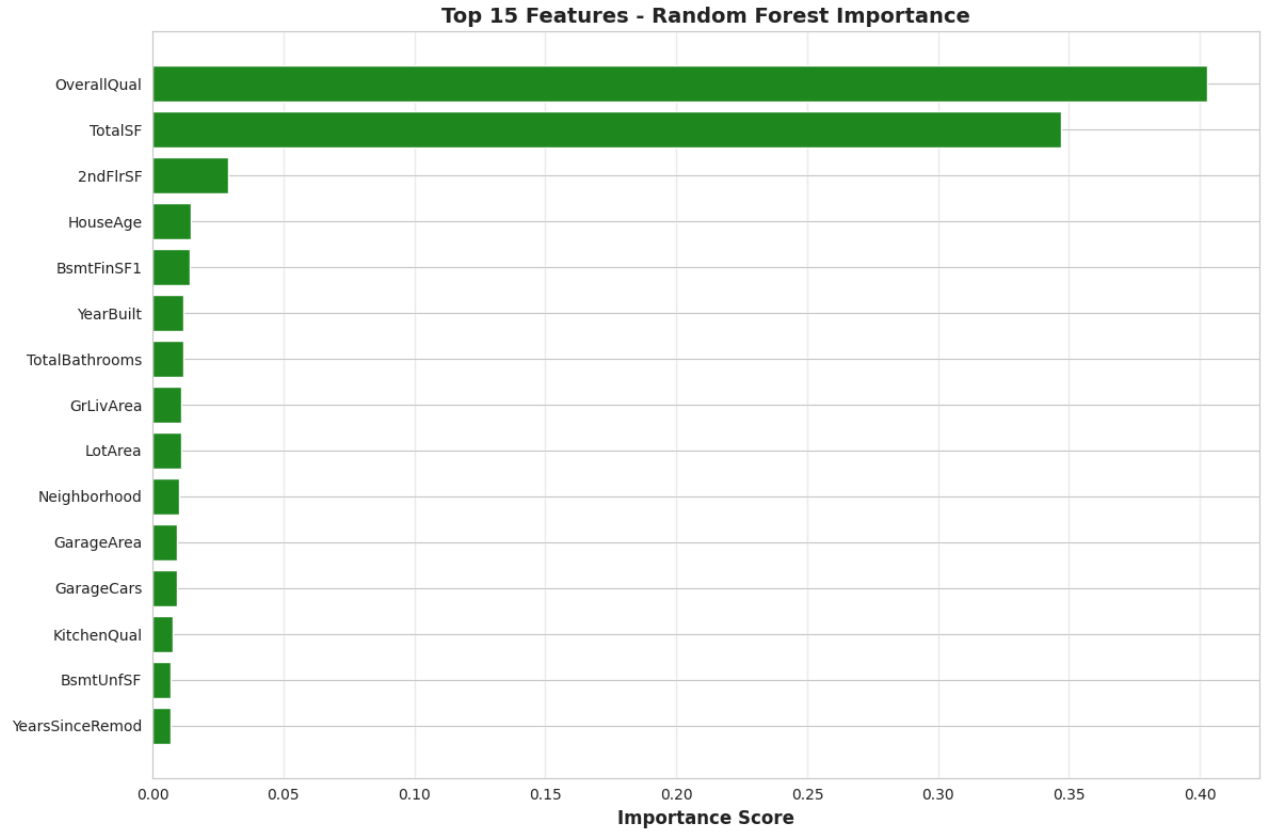

```
✅ Model comparison visualization complete
```

## 🔍 Step 14: Feature Importance / Input Impact Analysis

```
print("="*80)
print("🔍 FEATURE IMPORTANCE ANALYSIS")
print("="*80)

# Random Forest Feature Importance
print("\n📊 Random Forest - Feature Importance")
print("="*80)

rf_model = trained_models['Random Forest']
feature_importance_rf = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\nTop 15 Important Features (Random Forest):")
print(feature_importance_rf.head(15).to_string(index=False))

# Visualize Random Forest Feature Importance
plt.figure(figsize=(12, 8))
top_15_rf = feature_importance_rf.head(15)
plt.barh(range(len(top_15_rf)), top_15_rf['Importance'], color='forestgreen')
plt.yticks(range(len(top_15_rf)), top_15_rf['Feature'])
plt.xlabel('Importance Score', fontsize=12, fontweight='bold')
```

```
plt.title('Top 15 Features - Random Forest Importance', fontsize=14, fontweight='bold')
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

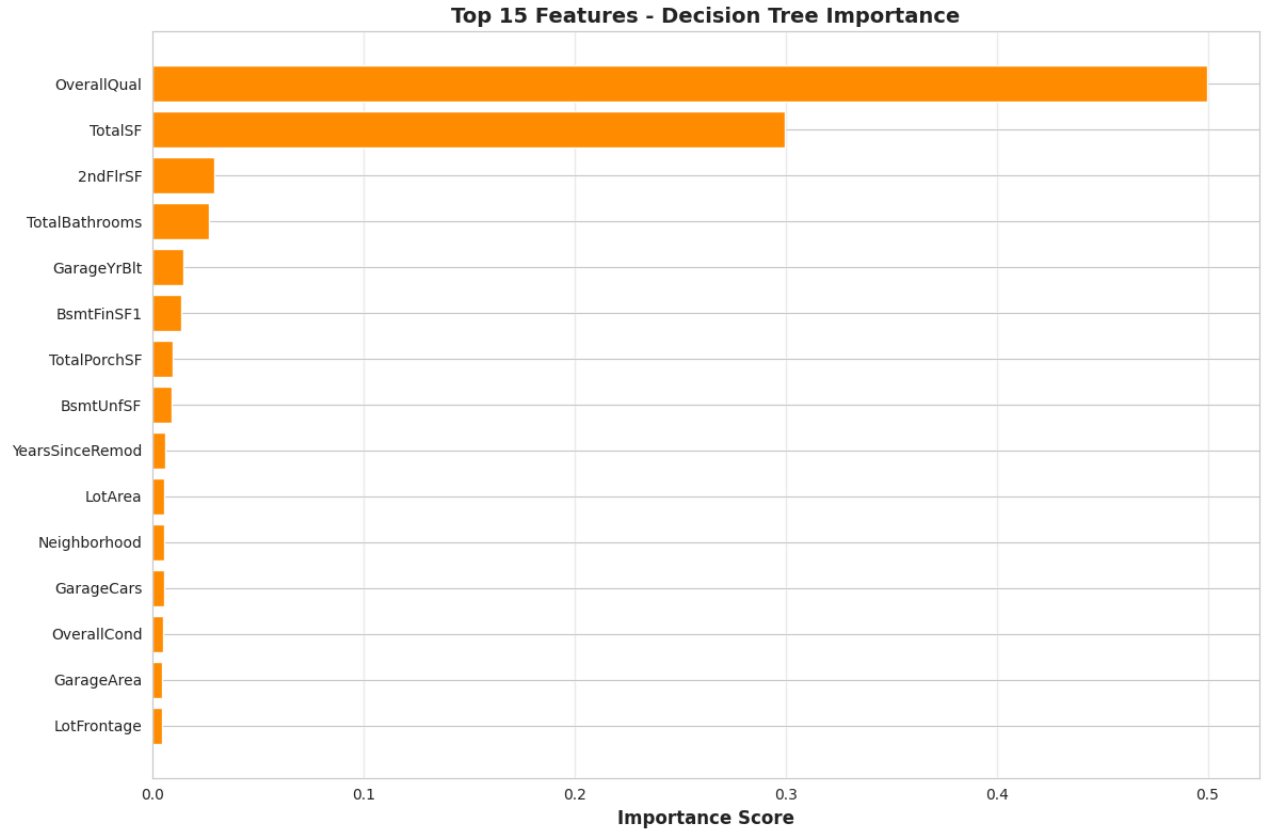
```
================================================================================
🔍 FEATURE IMPORTANCE ANALYSIS
================================================================================

📊 Random Forest - Feature Importance
================================================================================

Top 15 Important Features (Random Forest):
        Feature  Importance
    OverallQual    0.402579
        TotalSF    0.347126
       2ndFlrSF    0.028676
       HouseAge    0.014573
     BsmtFinSF1    0.014074
      YearBuilt    0.011727
 TotalBathrooms    0.011700
      GrLivArea    0.011115
        LotArea    0.010870
   Neighborhood    0.010009
     GarageArea    0.009318
     GarageCars    0.009249
    KitchenQual    0.007504
      BsmtUnfSF    0.006981
YearsSinceRemod    0.006685
```




```
# Decision Tree Feature Importance
print("\n" + "="*80)
print("📊 Decision Tree - Feature Importance")
print("="*80)

dt_model = trained_models['Decision Tree']
feature_importance_dt = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
}).sort_values('Importance', ascending=False)
```

```
print("\nTop 15 Important Features (Decision Tree):")
print(feature_importance_dt.head(15).to_string(index=False))

# Visualize Decision Tree Feature Importance
plt.figure(figsize=(12, 8))
top_15_dt = feature_importance_dt.head(15)
plt.barh(range(len(top_15_dt)), top_15_dt['Importance'], color='darkorange')
plt.yticks(range(len(top_15_dt)), top_15_dt['Feature'])
plt.xlabel('Importance Score', fontsize=12, fontweight='bold')
plt.title('Top 15 Features - Decision Tree Importance', fontsize=14, fontweight='bold')
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

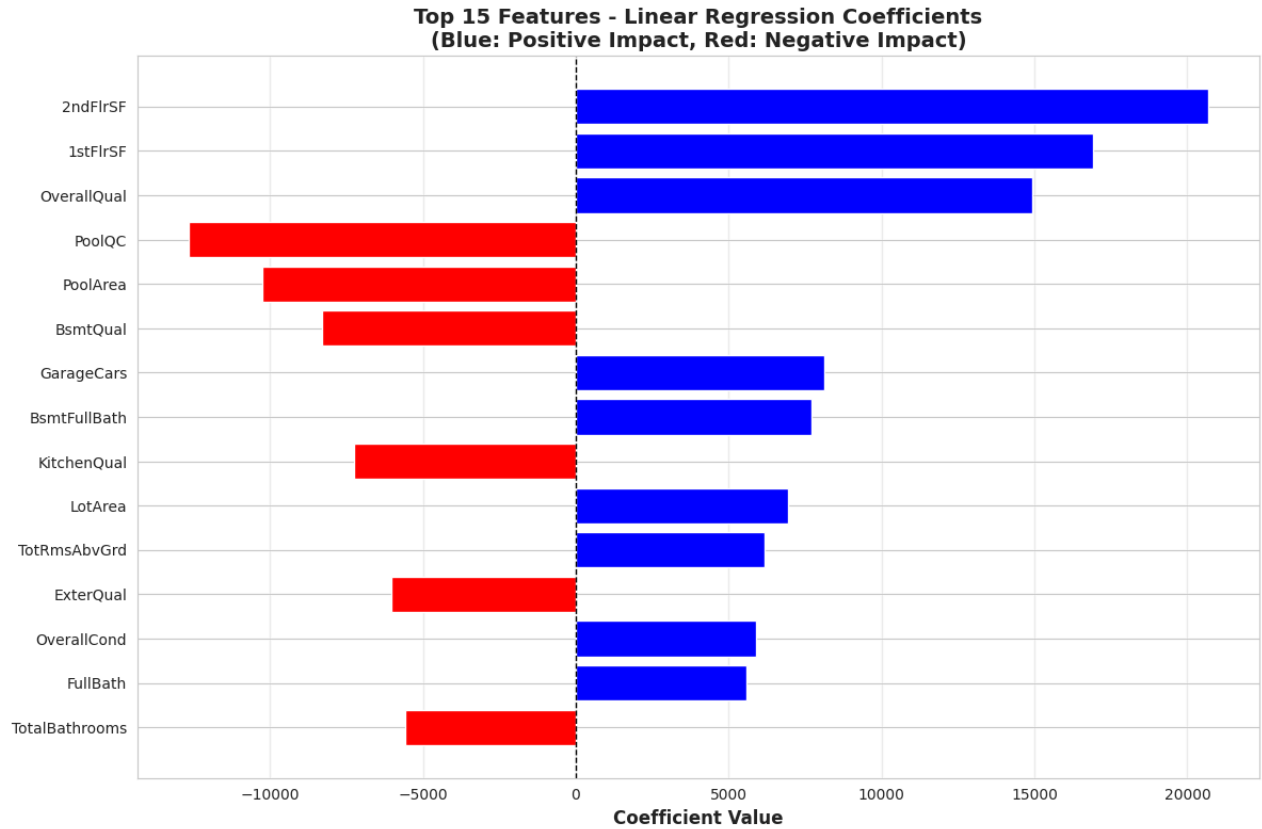
```
================================================================================
📊 Decision Tree - Feature Importance
================================================================================

Top 15 Important Features (Decision Tree):
        Feature  Importance
    OverallQual    0.499505
        TotalSF    0.299824
       2ndFlrSF    0.029307
 TotalBathrooms    0.026938
    GarageYrBlt    0.014532
     BsmtFinSF1    0.013715
   TotalPorchSF    0.009368
      BsmtUnfSF    0.009213
YearsSinceRemod    0.005802
        LotArea    0.005593
   Neighborhood    0.005354
     GarageCars    0.005334
    OverallCond    0.004763
     GarageArea    0.004487
    LotFrontage    0.004382
```




```
# Linear Regression Coefficients
print("\n" + "="*80)
print("📊 Linear Regression - Feature Coefficients")
print("="*80)
```

```
lr_model = trained_models['Linear Regression']
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lr_model.coef_
}).sort_values('Coefficient', key=abs, ascending=False)

print("\nTop 15 Features by Coefficient Magnitude (Linear Regression):")
print(coefficients.head(15).to_string(index=False))

# Visualize Linear Regression Coefficients
plt.figure(figsize=(12, 8))
top_15_coef = coefficients.head(15)
colors = ['red' if x < 0 else 'blue' for x in top_15_coef['Coefficient']]
plt.barh(range(len(top_15_coef)), top_15_coef['Coefficient'], color=colors)
plt.yticks(range(len(top_15_coef)), top_15_coef['Feature'])
plt.xlabel('Coefficient Value', fontsize=12, fontweight='bold')
plt.title('Top 15 Features - Linear Regression Coefficients\n(Blue: Positive Impact, Red: Negative Impact)',
          fontsize=14, fontweight='bold')
plt.axvline(x=0, color='black', linestyle='--', linewidth=1)
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
================================================================================
📊 Linear Regression - Feature Coefficients
================================================================================

Top 15 Features by Coefficient Magnitude (Linear Regression):
       Feature   Coefficient
      2ndFlrSF  20687.800765
      1stFlrSF  16931.333979
   OverallQual  14935.914092
        PoolQC -12664.842767
      PoolArea -10262.065797
      BsmtQual  -8296.009716
    GarageCars   8130.052323
  BsmtFullBath   7717.496496
   KitchenQual  -7246.920155
       LotArea   6932.468867
  TotRmsAbvGrd   6174.015407
     ExterQual  -6015.913070
   OverallCond   5888.644544
      FullBath   5587.335730
TotalBathrooms  -5567.203005
```




```
# Summary of Top Features Influencing Price
print("\n" + "="*80)
print("⭐ TOP FEATURES INFLUENCING PRICE")
print("="*80)

# Combine insights from different models
print("\n⏳ Most Important Features Across All Models:")
print("\nFrom Random Forest:")
print(feature_importance_rf.head(5)['Feature'].tolist())

print("\nFrom Decision Tree:")
print(feature_importance_dt.head(5)['Feature'].tolist())

print("\nFrom Linear Regression (by absolute coefficient):")
print(coefficients.head(5)['Feature'].tolist())

print("\n" + "="*80)
print("💡 KEY INSIGHTS:")
```

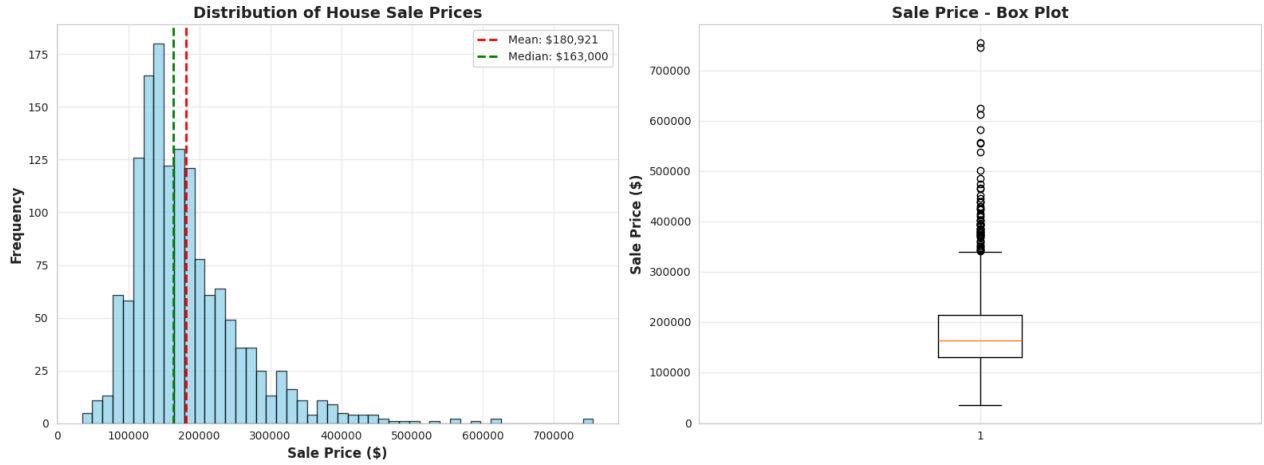
```
print("="*80)
print("The features listed above have the strongest influence on house prices.")
print("These should be prioritized when making pricing decisions.")
print("="*80)
```

```
================================================================================
⭐ TOP FEATURES INFLUENCING PRICE
================================================================================

🏆 Most Important Features Across All Models:

From Random Forest:
['OverallQual', 'TotalSF', '2ndFlrSF', 'HouseAge', 'BsmtFinSF1']

From Decision Tree:
['OverallQual', 'TotalSF', '2ndFlrSF', 'TotalBathrooms', 'GarageYrBlt']

From Linear Regression (by absolute coefficient):
['2ndFlrSF', '1stFlrSF', 'OverallQual', 'PoolQC', 'PoolArea']

================================================================================
💡 KEY INSIGHTS:
================================================================================
The features listed above have the strongest influence on house prices.
These should be prioritized when making pricing decisions.
================================================================================
```

## 📊 Step 15: Visualization - Data Analysis

```
print("="*80)
print("📊 DATA ANALYSIS VISUALIZATIONS")
print("="*80)

# 1. Target Variable Distribution (Histogram)
print("\n1️⃣ Target Variable Distribution")

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Histogram
axes[0].hist(y, bins=50, color='skyblue', edgecolor='black', alpha=0.7)
axes[0].set_xlabel('Sale Price ($)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Frequency', fontsize=12, fontweight='bold')
axes[0].set_title('Distribution of House Sale Prices', fontsize=14, fontweight='bold')
axes[0].axvline(y.mean(), color='red', linestyle='--', linewidth=2, label=f'Mean: ${y.mean():,.0f}')
axes[0].axvline(y.median(), color='green', linestyle='--', linewidth=2, label=f'Median: ${y.median():,.0f}')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Box plot
axes[1].boxplot(y, vert=True)
axes[1].set_ylabel('Sale Price ($)', fontsize=12, fontweight='bold')
axes[1].set_title('Sale Price - Box Plot', fontsize=14, fontweight='bold')
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Mean Price: ${y.mean():,.2f}")
print(f"Median Price: ${y.median():,.2f}")
print(f"Std Dev: ${y.std():,.2f}")
```

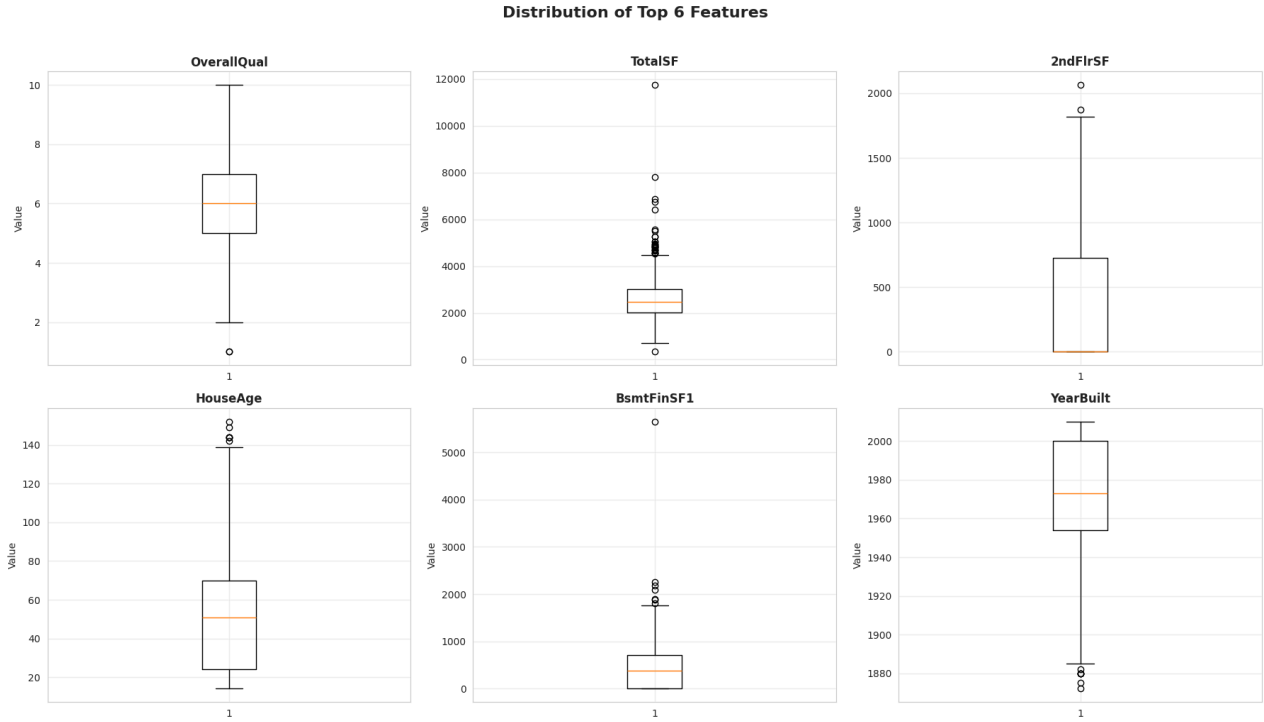
```
================================================================================
📊 DATA ANALYSIS VISUALIZATIONS
================================================================================

1️⃣ Target Variable Distribution
```




```
Mean Price: $180,921.20
Median Price: $163,000.00
Std Dev: $79,442.50
```

```
# 2. Boxplots for Key Numerical Features
print("\n" + "="*80)
print("2️⃣ Boxplots for Key Numerical Features")
print("="*80)

# Select top numerical features
numerical_features_plot = feature_importance_rf.head(6)['Feature'].tolist()
numerical_features_plot = [f for f in numerical_features_plot if f in X.columns]

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.ravel()

for idx, feature in enumerate(numerical_features_plot):
    axes[idx].boxplot(X[feature].dropna(), vert=True)
    axes[idx].set_title(f'{feature}', fontsize=12, fontweight='bold')
    axes[idx].set_ylabel('Value')
    axes[idx].grid(alpha=0.3)

plt.suptitle('Distribution of Top 6 Features', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print("✅ Boxplots generated for key features")
```

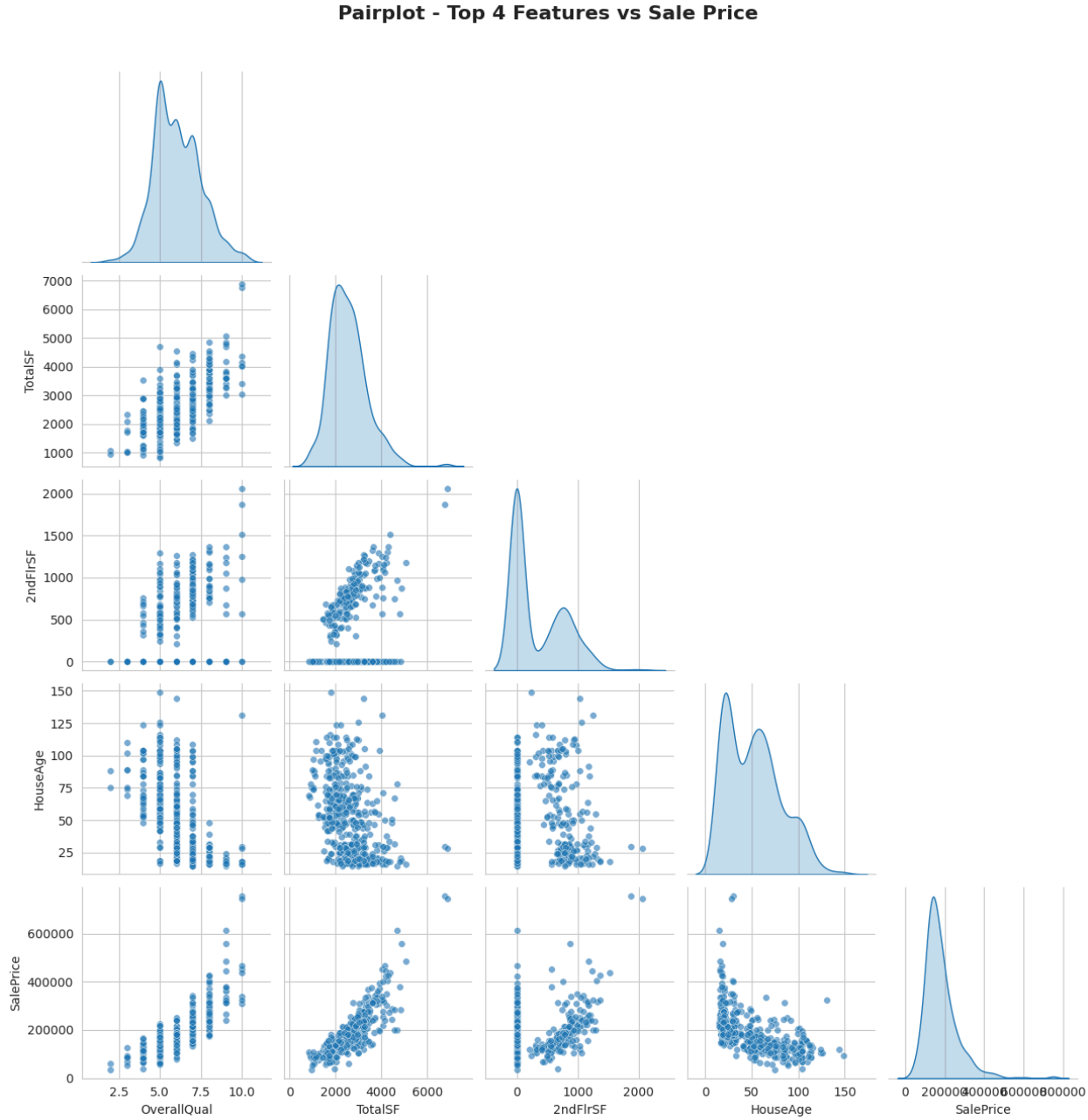
```
================================================================================
2️⃣ Boxplots for Key Numerical Features
================================================================================
```




```
✅ Boxplots generated for key features
```

```
# 3. Pairplot for Top Features (if dataset size allows)
print("\n" + "="*80)
print("3️⃣ Pairplot for Top Features")
print("="*80)

# Select top 4 features plus target for pairplot
top_4_features = feature_importance_rf.head(4)['Feature'].tolist()
pairplot_data = df_selected[top_4_features + ['SalePrice']].sample(min(500, len(df_selected)), random_state=42)

print(f"\nCreating pairplot with {len(pairplot_data)} samples...")

sns.pairplot(pairplot_data, diag_kind='kde', corner=True,
             plot_kws={'alpha': 0.6, 's': 30}, height=2.5)
plt.suptitle('Pairplot - Top 4 Features vs Sale Price', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print("✅ Pairplot generated")
```

```
================================================================================
3️⃣ Pairplot for Top Features
================================================================================

Creating pairplot with 500 samples...
```



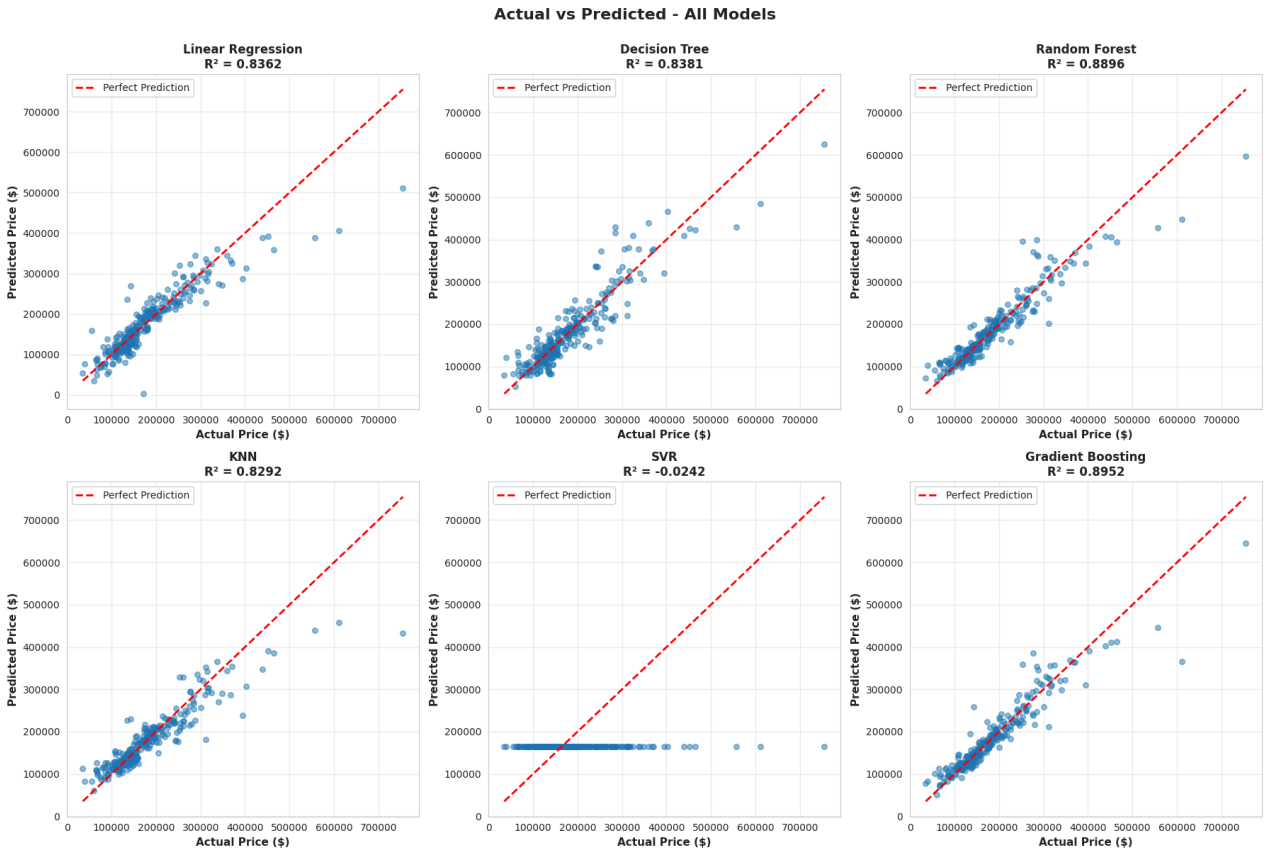

```
✅ Pairplot generated
```

## 📈 Step 16: Visualization - Model Performance

```
print("="*80)
print("📈 MODEL PERFORMANCE VISUALIZATIONS")
print("="*80)

# Get best model predictions
```

```
best_model = trained_models[best_model_name]
y_pred_best = predictions[best_model_name]

# 1. Actual vs Predicted Scatter Plot
print("\n1️⃣ Actual vs Predicted Values")

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

for idx, (name, y_pred) in enumerate(predictions.items()):
    axes[idx].scatter(y_test, y_pred, alpha=0.5, s=30)
    axes[idx].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                   'r--', lw=2, label='Perfect Prediction')
    axes[idx].set_xlabel('Actual Price ($)', fontsize=11, fontweight='bold')
    axes[idx].set_ylabel('Predicted Price ($)', fontsize=11, fontweight='bold')
    axes[idx].set_title(f'{name}\nR² = {comparison_df[comparison_df["Model"]==name]["R2 Score"].values[0]:.4f}',
                        fontsize=12, fontweight='bold')
    axes[idx].legend()
    axes[idx].grid(alpha=0.3)

plt.suptitle('Actual vs Predicted - All Models', fontsize=16, fontweight='bold', y=1.00)
plt.tight_layout()
plt.show()

print("✅ Actual vs Predicted plots generated")
```

```
================================================================================
📈 MODEL PERFORMANCE VISUALIZATIONS
================================================================================

1️⃣ Actual vs Predicted Values
```

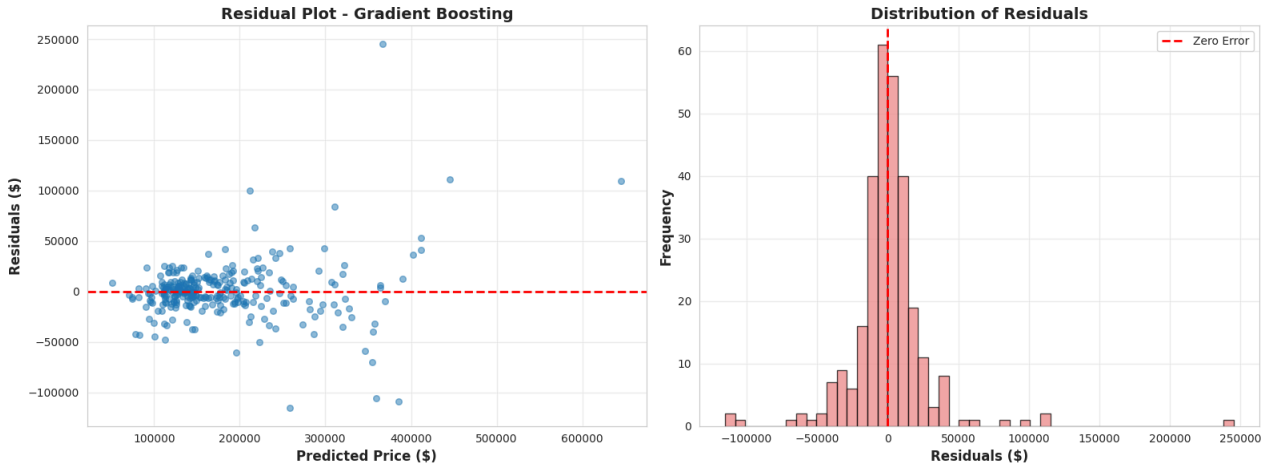



```
✅ Actual vs Predicted plots generated
```

```
# 2. Residual Plot (Best Model)
print("\n" + "="*80)
print("2️⃣ Residual Analysis (Best Model)")
print("="*80)

residuals = y_test - y_pred_best
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Residual plot
axes[0].scatter(y_pred_best, residuals, alpha=0.5, s=30)
axes[0].axhline(y=0, color='red', linestyle='--', linewidth=2)
axes[0].set_xlabel('Predicted Price ($)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Residuals ($)', fontsize=12, fontweight='bold')
axes[0].set_title(f'Residual Plot - {best_model_name}', fontsize=14, fontweight='bold')
axes[0].grid(alpha=0.3)

# Residual distribution
axes[1].hist(residuals, bins=50, color='lightcoral', edgecolor='black', alpha=0.7)
axes[1].axvline(x=0, color='red', linestyle='--', linewidth=2, label='Zero Error')
axes[1].set_xlabel('Residuals ($)', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Frequency', fontsize=12, fontweight='bold')
axes[1].set_title('Distribution of Residuals', fontsize=14, fontweight='bold')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Mean Residual: ${residuals.mean():,.2f}")
print(f"Std Dev of Residuals: ${residuals.std():,.2f}")
print("\n✅ Residual plots generated")
```

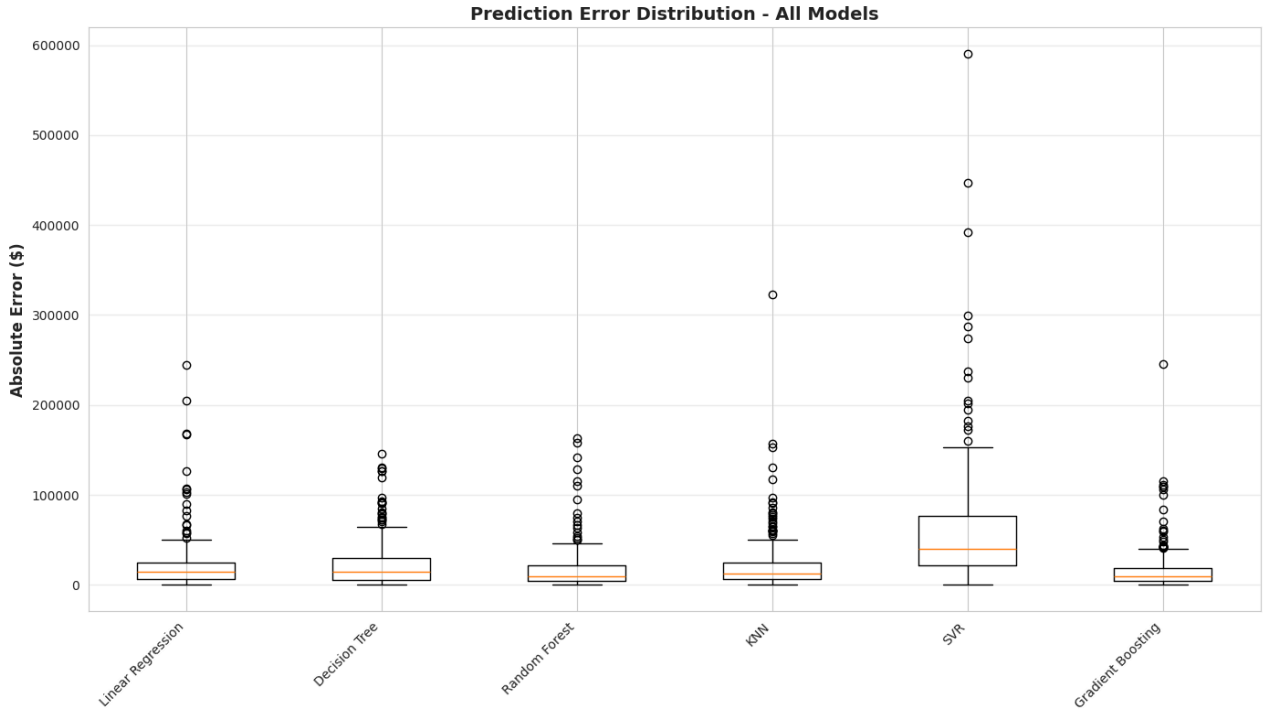
```
================================================================================
2️⃣ Residual Analysis (Best Model)
================================================================================
```




```
Mean Residual: $131.08
Std Dev of Residuals: $28,396.29

✅ Residual plots generated
```

```
# 3. Error Distribution across all models
print("\n" + "="*80)
print("3️⃣ Prediction Error Comparison")
print("="*80)

plt.figure(figsize=(14, 8))

error_data = []
labels = []
for name, y_pred in predictions.items():
    error_data.append(np.abs(y_test - y_pred))
    labels.append(name)

plt.boxplot(error_data, labels=labels)
plt.ylabel('Absolute Error ($)', fontsize=12, fontweight='bold')
plt.title('Prediction Error Distribution - All Models', fontsize=14, fontweight='bold')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

print("✅ Error distribution plot generated")
```

```
================================================================================
3️⃣ Prediction Error Comparison
================================================================================
```




```
✅ Error distribution plot generated
```

## 📐 Step 17: Model Parameters (Equation)

```
print("="*80)
print("📐 LINEAR REGRESSION MODEL EQUATION")
print("="*80)

lr_model = trained_models['Linear Regression']

print("\n📊 Linear Regression Equation:")
print("="*80)
print(f"\nIntercept (β₀): ${lr_model.intercept_:,.2f}")
print("\nCoefficients (β₁, β₂, ..., βₙ):")
print("-" * 80)

# Show top 10 coefficients
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lr_model.coef_
}).sort_values('Coefficient', key=abs, ascending=False)

print("\nTop 10 Coefficients (by absolute value):")
for idx, row in coef_df.head(10).iterrows():
    sign = '+' if row['Coefficient'] >= 0 else ''
    print(f"{row['Feature']:30s}: {sign}{row['Coefficient']:12,.2f}")

print("\n" + "="*80)
print("📝 EQUATION INTERPRETATION:")
print("="*80)
print("\nPrice = Intercept + (Coefficient₁ × Feature₁) + (Coefficient₂ × Feature₂) + ... + Error")
print("\n💡 Interpretation:")
print("   • Positive coefficient: Increase in feature → Increase in price")
print("   • Negative coefficient: Increase in feature → Decrease in price")
print("   • Larger absolute value: Stronger impact on price")
print("\n📌 Example:")
top_feature = coef_df.iloc[0]
print(f"   If '{top_feature['Feature']}' increases by 1 unit,")
if top_feature['Coefficient'] > 0:
    print(f"   the predicted price increases by approximately ${abs(top_feature['Coefficient']):,.2f}")
```